

RELATÓRIO EXPERIMENTO 4

Nome: Renato Gabriel Lima da Silva

Matrícula: 251027871

INTRODUÇÃO

Neste experimento, iremos abordar algumas entidades que são funções booleanas, entretanto iremos implementá-las a partir de multiplexadores e decodificadores na linguagem de descrição de hardware VHDL e simular no software ModelSim.

TEORIA

1 - Questão 1

Na primeira questão devemos implementar uma entidade com 3 bits de entradas (A,B e C) e 2 bits de saídas (X e Y), que implemente a seguinte função booleana:

$$X = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot B$$

$$Y = \bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + A \cdot B \cdot C$$

Que possui a seguinte tabela verdade:

<i>A</i>	<i>B</i>	<i>C</i>	<i>X</i>	<i>Y</i>
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

Tabela 1: tabela verdade das funções X e Y.

Para esse exercício, deve usar apenas 2 multiplexadores de 4 entradas e uma porta inversora. Devido a essas restrições, será necessário introduzir uma variável nas saídas. Ou seja, transformá-las em funções de uma certa variável. Pode-se fazer

isso agrupando-se de duas em duas linhas da Tabela 1 e compará-las com os valores de C (o bit de menor significância). O resultado é a Tabela 2

A	B	$X(C)$	$Y(C)$
0	0	0	1
0	1	C	$\overline{C}$
1	0	$\overline{C}$	0
0	0	1	C

Tabela 2: introdução da variável C

Com a Tabela 2, é possível usar um multiplexador 4x1 para implementar a saída X e outro para a saída Y. A porta lógica NOT será utilizada para inverter o sinal de C quando necessário. Para facilitar a implementação fazemos um esquema visual de como fica a implementação do circuito com o multiplexador:

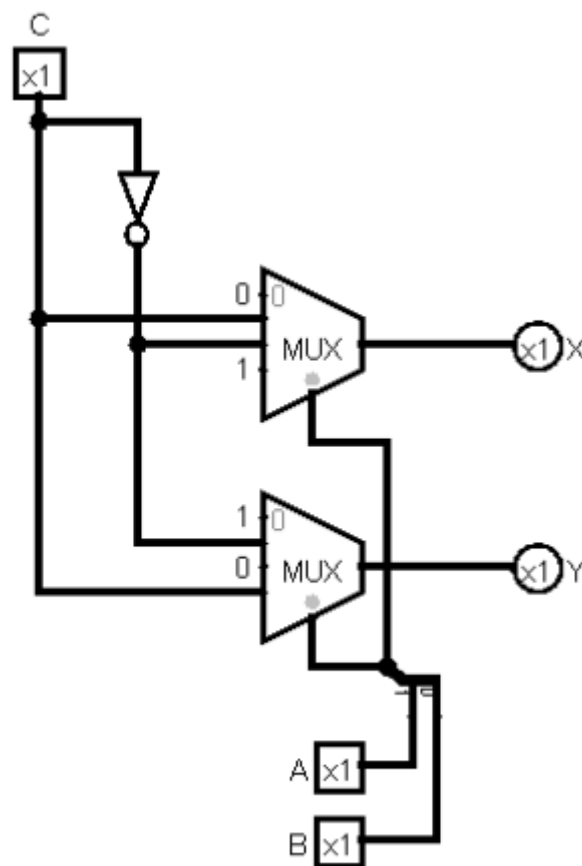


Imagem 1: Circuito da questão 1.

2 - Questão 2

Na segunda questão devemos implementar uma entidade com 7 bits de entradas (A,B,C,D,E,F e G) e 1 bits de saída (S), que implemente a seguinte função booleana:

$$S = F \cdot G + A \cdot B \cdot C \cdot D \bar{E} \cdot \bar{F} \cdot G + \bar{A} \cdot \bar{B} \cdot \bar{C} \cdot \bar{D} \cdot \bar{E} \cdot \bar{F} \cdot G + A \cdot \bar{B} \cdot C \cdot E \cdot F \bar{G} + \bar{A} \cdot B \cdot C \cdot D \cdot \bar{E} \cdot F \cdot \bar{G} + A \cdot B \cdot C \cdot D \cdot E \cdot \bar{F} \cdot \bar{G} + A \cdot \bar{B} \cdot \bar{C} \cdot D \cdot E \cdot \bar{F} \cdot \bar{G}$$

Que possui a seguinte tabela verdade:

A	B	C	D	E	F	G	Z
x	x	x	x	x	1	1	1
1	1	1	1	1	0	0	1
1	1	1	1	0	0	1	1
1	0	1	x	1	1	0	1
1	0	0	1	1	0	0	1
0	1	1	1	0	1	0	1
0	0	0	0	0	0	1	1
Outros Valores							0

Tabela 3: tabela verdade da questão 2 (S = Z) (X = don't care)

Para esse exercício, deve usar apenas um decodificador 4x16 e um multiplexador 8x1 e três portas OU.

Seguindo as restrições, prova-se essencial a divisão do sistema em duas partes distintas. A primeira, consistindo dos bits de entrada A, B, C, D serão utilizados no decodificador para a seleção de mintermos desejados, enquanto os bits restantes, E, F, G serão usados no multiplexador para completar o sistema. Pode-se usar da Tabela 3 para construir uma nova tabela que indica quais mintermos serão utilizados em quais entradas do decodificador.

Obteve-se, então, a Tabela 4, que será posteriormente utilizada para a construção do código do sistema:

E	F	G	Entradas no mux
0	0	0	0
0	0	1	$ABCD + \bar{A}\bar{B}\bar{C}\bar{D}$
0	1	0	$\bar{A}BCD$
0	1	1	1
1	0	0	$ABCD + A\bar{B}\bar{C}\bar{D}$
1	0	1	0
1	1	0	$A\bar{B}CD + A\bar{B}C\bar{D}$
1	1	1	1

Tabela 4: tabela com o mux,decoder e portas ou.

Para facilitar a implementação fazemos um esquema visual de como fica a implementação do circuito com o decodificador, multiplexador e as portas OU:

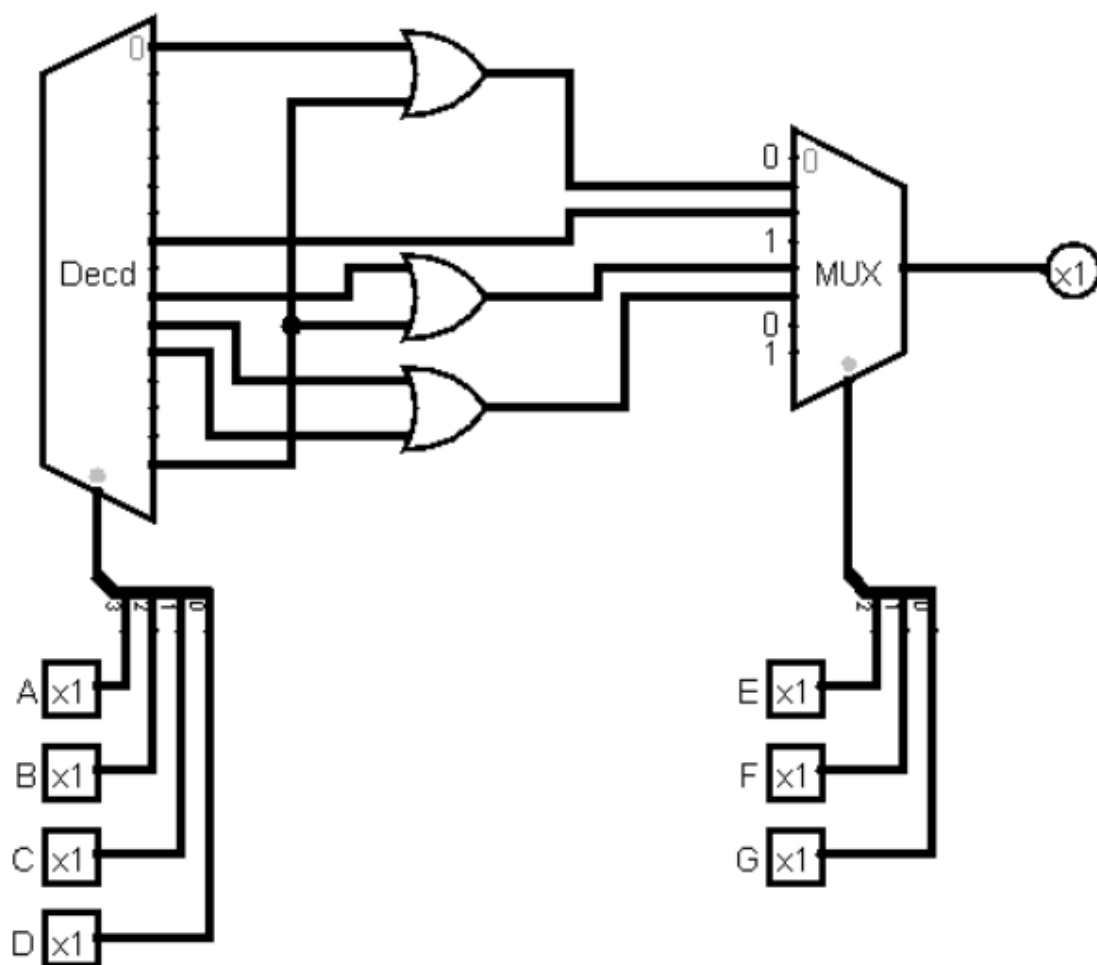


Imagem 2 : Circuito da questão 2.

CÓDIGOS

1 - Questão 1

Agora vamos analisar a implementação do código, antes de partirmos para o código principal é necessário a implementação dos outros circuitos, que serão utilizados para a implementação das funções, sendo elas o multiplexador 4x1 e a porta inversora:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity multiplexador is
5      port(S: in std_logic_vector (1 downto 0);
6           D: in std_logic_vector (3 downto 0);
7           Y: out std_logic);
8  end multiplexador;
9
10 architecture multiplexador_arch of multiplexador is
11 begin
12     Y<= (D(0)and(not S(1))and(not S(0))) or (D(1)and(not S(1))and S(0)) or
13         (D(2) and S(1) and(not S(0))) or (D(3) and S(1) and S(0));
14 end multiplexador_arch;
```

Listagem 1: código do mux 4x1

O código da Listagem 1 implementa os requisitos para a funcionalidade padrão de arquivos VHDL, e, então, usa dois vetores para representar a entrada e um bit para a saída. Um vetor S (set) que indica qual bit do vetor D (data) será representado na saída Y.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- Porta logica NOT
5  entity gatenot is
6      port (
7          A: in std_logic;
8          Y: out std_logic
9      );
10 end gatenot;
11
12 architecture gatenot_arch of gatenot is
13 begin
14     Y <= not A;
15 end gatenot_arch;
```

Listagem 2: código da porta lógica NOT

O código da Listagem 2 implementa a funcionalidade padrão de códigos VHDL, e procede para especificar um circuito que recebe uma entrada A e retorna seu inverso pela saída Y .

Com isso possuímos os componentes necessários para implementar as duas funções, portanto, com base na tabela 2 e da imagem 1 sigamos para o código da questão 1:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity questaol is
5      port(A,B,C: in std_logic;
6           X,Y: out std_logic);
7  end questaol;
8
9  architecture questaol_arch of questaol is
10
11      component multiplexador is
12          port(S: in std_logic_vector (1 downto 0);
13              D: in std_logic_vector (3 downto 0);
14              Y: out std_logic);
15      end component;
16
17      component gatenot is
18          port (
19              A: in std_logic;
20              Y: out std_logic
21          );
22      end component;
23
24      signal setvect      : std_logic_vector(1 downto 0);
25      signal datal, data2 : std_logic_vector(3 downto 0);
26      signal notC         : std_logic;
27
28      begin
29          setvect(0) <= B;
30          setvect(1) <= A;
31
32          INT1 : gatenot port map(C, notC);
33
34          datal(0) <= '0';
35          datal(1) <= C;
36          datal(2) <= notC;
37          datal(3) <= '1';
38
39          data2(0) <= '1';
40          data2(1) <= notC;
41          data2(2) <= '0';
42          data2(3) <= C;
43
44          INT2 : multiplexador port map(setvect, data2, Y);
45          INT3 : multiplexador port map(setvect, datal, X);
46      end questaol_arch;
```

Listagem 3: código questão 1

Nas linhas 4 a 7, define-se as entradas A, B e C e as saídas X e Y , respectivamente. Todas são bits simples de entrada e saída. Logo em seguida, começa-se a definir a arquitetura do sistema. Primeiramente, deve-se declarar os componentes que serão utilizados: outros circuitos que serão importados para serem implementados no sistema 1. Adiciona-se, então, dentro do bloco da arquitetura questão1_arch as entidades apresentados anteriormente mas agora com o nome component no lugar de entity (linha 11 a 22).

Após os componentes colocamos alguns sinais que vão ser os fios internos que serão utilizados para conexões entre componentes (linhas 24 a 26). Serão necessários três vetores e um bit para uso interno. O vetor de 2 bits setvect listado na linha 1 da Listagem 5 será usado como a entrada S do multiplexador 4x1. Já os vetores de 4 bits data1 e data2 listados na linha 2 serão utilizados como a entrada D do multiplexador 4x1 para simular as saídas X e Y , respectivamente. O sinal de bit simples notC listado na linha 3 representará o inverso da entrada C.

Por fim, ainda dentro da arquitetura, inicia-se a lógica do sistema (linha 28 a 46), correlacionando a Tabela 2 com os vetores necessários, e finalmente introduzindo-os aos multiplexadores para obter-se o resultado desejado.

De início, adiciona-se os bits A e B às suas devidas posições no vetor que será utilizado como entrada S dos multiplexadores, . Após definir o inverso de C na linha 32, usamos os vetores data1 e data2 para implementar X e Y, respectivamente. Finalmente, nas linhas 44 e 45, chamamos o código do multiplexador para produzir as saidas desejadas.

2 - Questão 2

Agora vamos analisar a implementação do código da questão 2, antes de partirmos para o código principal é necessário a implementação dos outros circuitos, que serão utilizados para a implementação das funções, sendo elas o multiplexador 8x1, o decodificador 4X16 e a porta OU:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  entity MUX8X1 is
6      port (S: in std_logic_vector(2 downto 0);
7            D: in std_logic_vector(7 downto 0);
8            Y: out std_logic);
9  end MUX8X1;
10
11  architecture MUX8X1_arch of MUX8X1 is
12  begin
13      Y <= D(0) when (S = "000") else
14            D(1) when (S = "001") else
15            D(2) when (S = "010") else
16            D(3) when (S = "011") else
17            D(4) when (S = "100") else
18            D(5) when (S = "101") else
19            D(6) when (S = "110") else
20            D(7);
21  end MUX8X1_arch;
```

Listagem 4: código do mux 8x1

O código da Listagem 4 implementa o multiplexador 8x1 muito semelhante ao apresentado anteriormente porém com 3 bits de seleção e 8 de dados.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  entity DECOD4X16 is
6      port (A: in std_logic_vector(3 downto 0);
7            Y: out std_logic_vector(15 downto 0));
8  end DECOD4X16;
9
10
11  architecture DECOD4X16_arch of DECOD4X16 is
12  begin
13      with A select
14          Y <= "0000000000000001" when "0000",
15              "0000000000000010" when "0001",
16              "0000000000000100" when "0010",
17              "0000000000001000" when "0011",
18              "0000000000010000" when "0100",
19              "0000000000100000" when "0101",
20              "0000000001000000" when "0110",
21              "0000000010000000" when "0111",
22              "0000000100000000" when "1000",
23              "0000001000000000" when "1001",
24              "0000010000000000" when "1010",
25              "0000100000000000" when "1011",
26              "0001000000000000" when "1100",
27              "0010000000000000" when "1101",
28              "0100000000000000" when "1110",
29              "1000000000000000" when others;
30  end DECOD4X16_arch;

```

Listagem 5: código do decodificador 4X16

O decodificador recebe apenas uma entrada, o vetor A de 4 bits que determina qual dos 16 bits disponíveis no vetor de saída Y deve ser ligada. Por exemplo, ao enviar o vetor 0101 como entrada A, a saída Y5 será acesa, enquanto as outras permanecem como 0.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- Porta logica OR
5  entity GATEOR is
6      port (
7          A: in std_logic;
8          B: in std_logic;
9          Y: out std_logic
10      );
11  end GATEOR;
12
13  architecture GATEOR_ARCH of GATEOR is
14  begin
15      Y <= A or B;
16  end GATEOR_ARCH;

```

Listagem 6: código da porta lógica OR

A porta OR recebe duas entradas simples A e B, com apenas uma saída simples Y . O resultado do circuito será 1 se qualquer uma das duas entradas forem 1.

Com isso, possuímos os componentes necessários para implementar a função, portanto, com base na tabela 4 e da imagem 2 sigamos para o código da questão 2:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity questao2 is
5      port (
6          A: in std_logic;
7          B: in std_logic;
8          C: in std_logic;
9          D: in std_logic;
10         E: in std_logic;
11         F: in std_logic;
12         G: in std_logic;
13         Z: out std_logic
14     );
15 end questao2;
16
17 architecture questao2_arch of questao2 is
18
19     component MUX8X1 is
20         port (
21             S: in std_logic_vector(2 downto 0);
22             D: in std_logic_vector(7 downto 0);
23             Y: out std_logic
24         );
25     end component;
26
27     component GATEOR is
28         port (
29             A: in std_logic;
30             B: in std_logic;
31             Y: out std_logic
32         );
33     end component;
34
35     component DECOD4X16 is
36         port (
37             A: in std_logic_vector(3 downto 0);
38             Y: out std_logic_vector(15 downto 0)
39         );
40     end component;
41
42     signal decvect      : std_logic_vector(3 downto 0);
43     signal setvect      : std_logic_vector(2 downto 0);
44     signal decresult    : std_logic_vector(15 downto 0);
45     signal datain       : std_logic_vector(7 downto 0);
46
```

```

47  signal wire1, wire2, wire3 : std_logic;
48
49  begin
50      decvect(3) <= A;
51      decvect(2) <= B;
52      decvect(1) <= C;
53      decvect(0) <= D;
54
55      setvect(2) <= E;
56      setvect(1) <= F;
57      setvect(0) <= G;
58
59      INT1 : DECOD4X16 port map(decvect, decresult);
60
61      INT2 : GATEOR port map(decresult(0), decresult(15), wire1);
62      INT3 : GATEOR port map(decresult(15), decresult(9), wire2);
63      INT4 : GATEOR port map(decresult(10), decresult(11), wire3);
64
65      datain(0) <= '0';
66      datain(1) <= wire1;
67      datain(2) <= decresult(7);
68      datain(3) <= '1';
69      datain(4) <= wire2;
70      datain(5) <= '0';
71      datain(6) <= wire3;
72      datain(7) <= '1';
73
74      INT5 : MUX8X1 port map(setvect, datain, Z);
75
76  end questao2_ARCH;

```

Listagem 7: código questão 2

A implementação será dividida em três etapas. Primeiro, usa-se o decodificador nos bits de entrada A, B, C e D. Então, seleciona-se os mintermos relevantes e agrupa-os com portas OR. Finalmente, usando um multiplexador com os valores E, F e G como bits seletores, conectando as entradas data de acordo com a Tabela 4.

Nas linhas 4 a 15, as entradas e a saída estão declaradas. Segue-se para a montagem da arquitetura do sistema. Começa-se introduzindo os circuitos previamente implementados no sistema como componentes igual na questão anterior (linha 17 a 40).

Em seguida, são definidos os sinais que serão utilizados na questão. Precisa-se de vetores para as entradas e saídas do multiplexador e decodificador, além de fios para conectar as portas OR em suas devidas posições (linha 42 a 47). O vetor decvect definido na linha 42 será utilizado como a entrada do decodificador, o vetor setvect da linha 43 como entrada set do multiplexador, o decresult da linha 44 como saída do decodificador e o vetor datain da linha 45 como entrada data do multiplexador. Os fios wire1, wire2 e wire3 da linha 47 serão utilizados para implementar as portas OR.

Nas linhas 50 a 53, associa-se o valor de entrada A no valor de maior significância do vetor decvect, B no segundo, e assim por diante para os valores C e D. Realiza-se o mesmo, nas linhas 55 a 57, para o vetor setvect e as entradas E, F e G, respectivamente. Na linha 59, declara-se o uso do circuito do decodificador, associando-se o vetor decvect como sua entrada, e decresult como sua saída.

Em seguida, realiza-se operações OR a fim de modelar os mintermos necessários para as conexões no multiplexador. Na linha 61, realiza-se a soma lógica do mintermo AsBsCsDs (valor 0 do decodificador) com o mintermo ABCD (valor 15) e envia-se o resultado para o fio wire1. Na linha 62, associa-se os mintermos ABCD (15) e ABC s Ds (9) com um OR para o fio wire2. Por fim, associa-se ABCD s (11) com ABCs Ds (10) no fio wire3.

Usando o vetor datain cria-se a entrada data que será utilizada no multiplexador, logo iniciando-se suas associações de acordo com a Tabela 4. Nas entradas 0 e 5 (linhas 65 e 70, respectivamente), insere-se uma constante baixa. Já nas 3 e 7 (linhas 68 e 72) insere-se uma constante alta. Para a entrada 1, na linha 66, usa-se o wire1, na entrada 4, na linha 69, o wire2 e, na entrada 6, na linha 71, o wire3. Por último, diretamente conecta-se o mintermo ABCD s do decodificador na entrada 2, na linha 67.

Para concluir o sistema 2, cria-se o multiplexador na linha 74: associa-se como entrada data o vetor datain, como entrada set o vetor setvect e como saída o bit Z. Assim, o sistema está completamente implementado.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

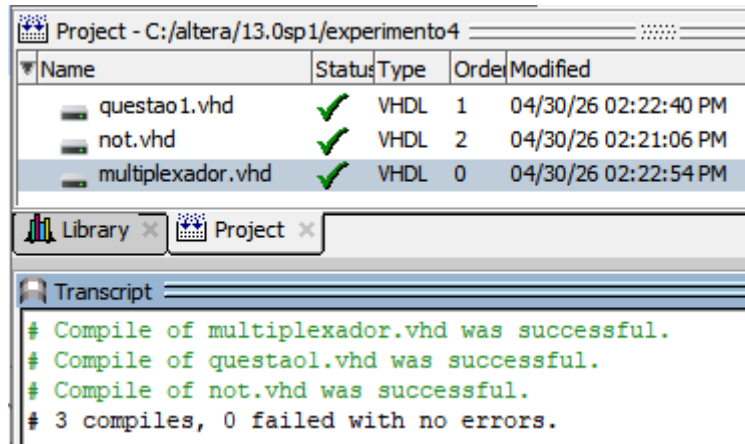


Figura 1: Compilação dos códigos da questão 1.

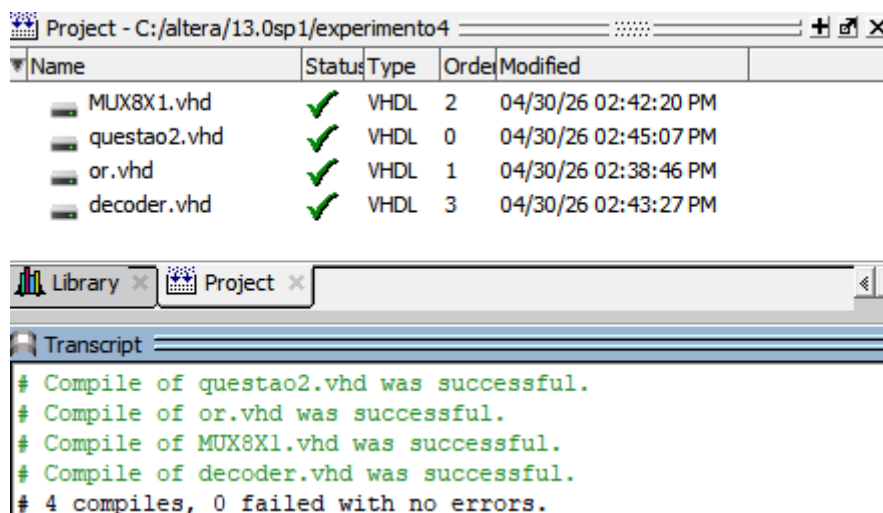


Figura 2: Compilação dos códigos da questão 2.

Como visto na figura 1 e 2, nenhum código tem algum erro de sintaxe.

SIMULAÇÃO E ANÁLISE

1- QUESTÃO 1

Para verificar a correção do código implementado, ele foi simulado no software Modelsim. Para a simulação, foi feito um testbench que testa todas as combinações possíveis para A,B e C. Segue o código do testbench:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity questao1_tb is
6  end questao1_tb;
7
8  architecture tb_arch of questao1_tb is
9
10     signal t_A, t_B, t_C : std_logic;
11     signal t_X, t_Y      : std_logic;
12
13 begin
14
15     UUT: entity work.questao1
16         port map (
17             A => t_A,
18             B => t_B,
19             C => t_C,
20             X => t_X,
21             Y => t_Y
22         );
23
24     stim_proc: process
25     begin
26         for i in 0 to 7 loop
27             (t_A, t_B, t_C) <= std_logic_vector(to_unsigned(i, 3));
28             wait for 10 ns;
29         end loop;
30         wait;
31     end process;
32
33 end tb_arch;

```

Listagem 8: Código do testbench da questão 1.

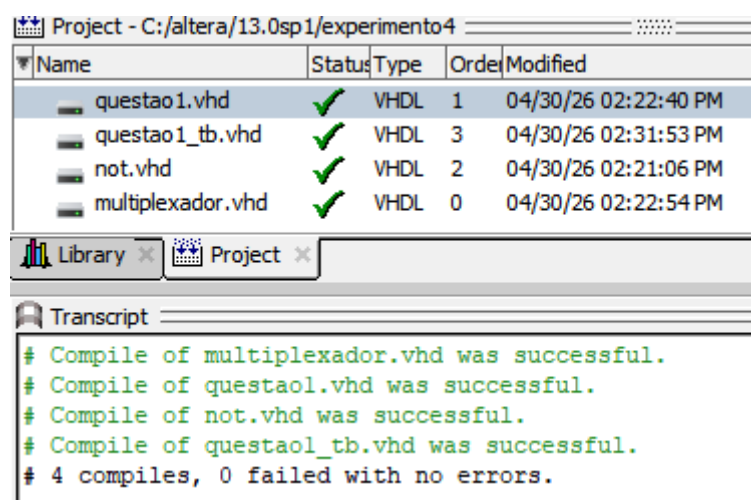


Figura 3: compilação com o testbench.

Com isso conseguimos simular por meio do ModelSim e observar os resultados por meio da forma de onda apresentada:

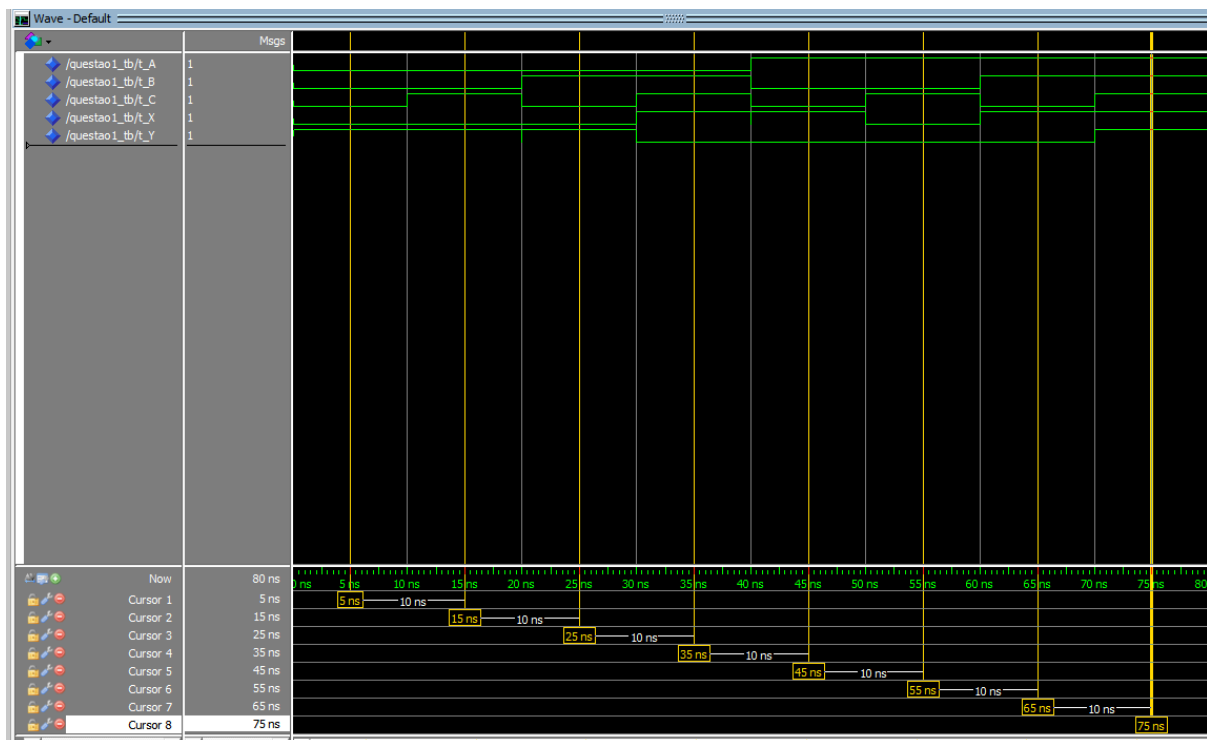


Figura 4: Simulação no ModelSim da questão 1.

Cada cursor está em um valor de A,B e C e suas respectivas saídas X e Y ficando da seguinte forma:

- Cursor 1: A = 0, B = 0, C = 0, X = 0 e Y = 1
- Cursor 2: A = 0, B = 0, C = 1, X = 0 e Y = 1
- Cursor 3: A = 0, B = 1, C = 0, X = 0 e Y = 1
- Cursor 4: A = 0, B = 1, C = 1, X = 1 e Y = 0
- Cursor 5: A = 1, B = 0, C = 0, X = 1 e Y = 0
- Cursor 6: A = 1, B = 0, C = 1, X = 0 e Y = 0
- Cursor 7: A = 1, B = 1, C = 0, X = 1 e Y = 0
- Cursor 8: A = 1, B = 1, C = 1, X = 1 e Y = 1

Portanto ficou igual a tabela verdade apresentada anteriormente na Tabela 1.

2 - QUESTÃO 2

Para verificar a correção do código implementado, ele foi simulado no software Modelsim. Para a simulação, foi feito um testbench que testa todas as combinações possíveis de entrada. Segue o código do testbench:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity questao2_tb is
6  end questao2_tb;
7
8  architecture tb_arch of questao2_tb is
9
10     signal t_A, t_B, t_C, t_D, t_E, t_F, t_G : std_logic;
11     signal t_Z : std_logic;
12     signal t_comb : std_logic_vector(6 downto 0);
13
14 begin
15     UUT: entity work.questao2
16         port map (
17             A => t_A, B => t_B, C => t_C, D => t_D,
18             E => t_E, F => t_F, G => t_G,
19             Z => t_Z
20         );
21     stim_proc: process
22     begin
23         for i in 0 to 127 loop
24             t_comb <= std_logic_vector(to_unsigned(i, 7));
25
26             t_A <= t_comb(6);
27             t_B <= t_comb(5);
28             t_C <= t_comb(4);
29             t_D <= t_comb(3);
30             t_E <= t_comb(2);
31             t_F <= t_comb(1);
32             t_G <= t_comb(0);
33
34             wait for 10 ns;
35         end loop;
36
37         wait;
38     end process;
39 end tb_arch;
```

Listagem 9: Código do testbench da questão 2

O código usa um sinal auxiliar chamado `t_comb` de 7 bits para facilitar a contagem. A cada 10ns, ele "quebra" esse vetor e distribui os bits para as entradas individuais:

- `t_A` recebe o bit mais significativo (MSB), o bit 6.
- `t_G` recebe o bit menos significativo (LSB), o bit 0.

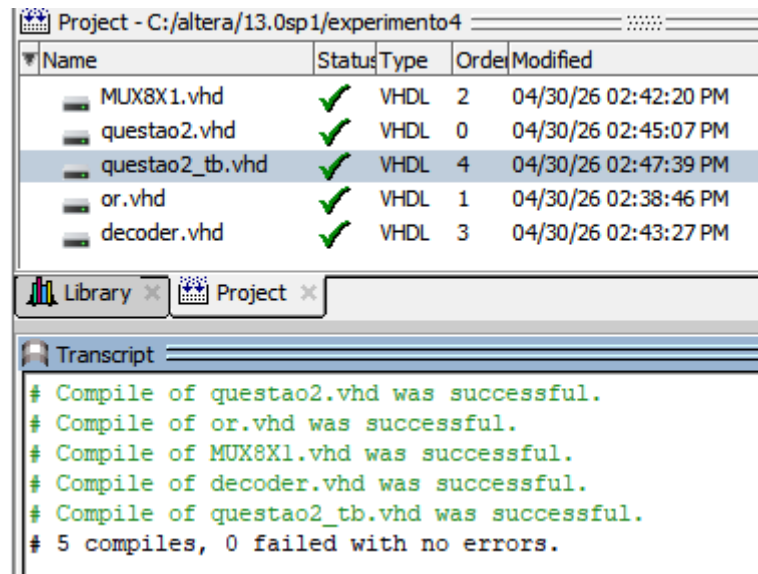


Figura 5: compilação com o testbench.

Com isso conseguimos simular por meio do ModelSim e observar os resultados por meio da forma de onda apresentada:

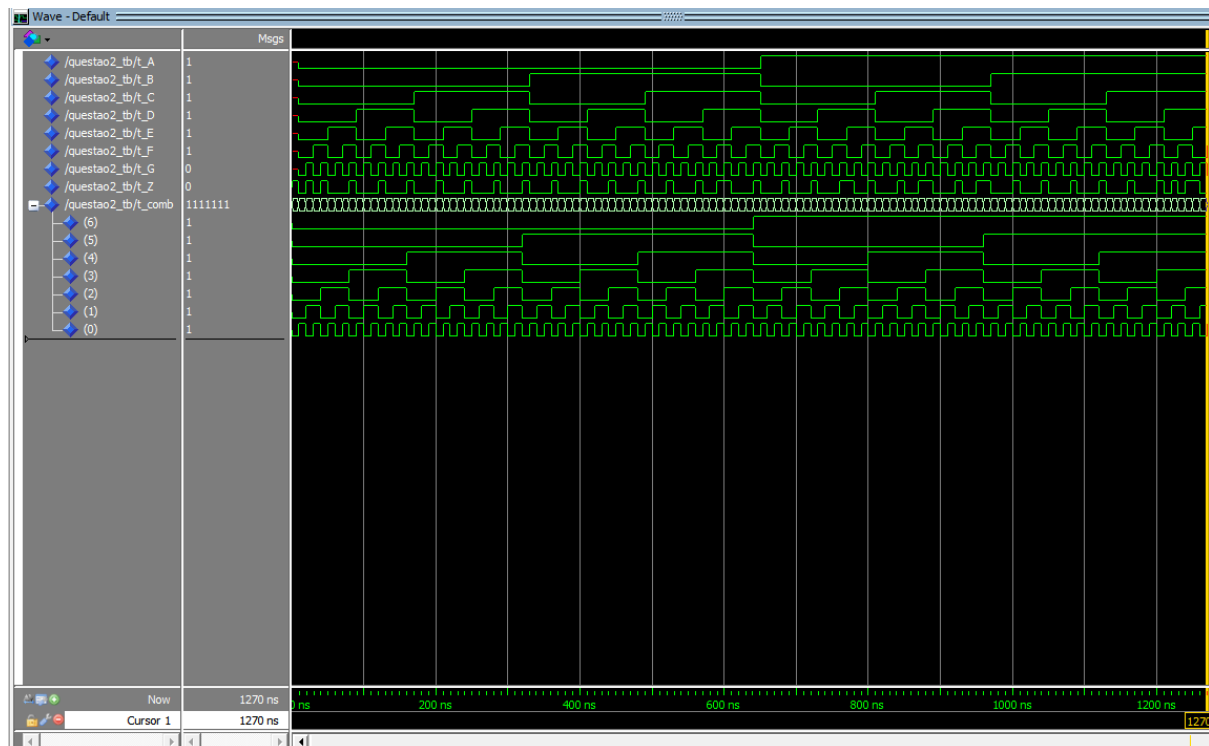


Figura 5: Simulação de todos os casos no ModelSim da questão 2.

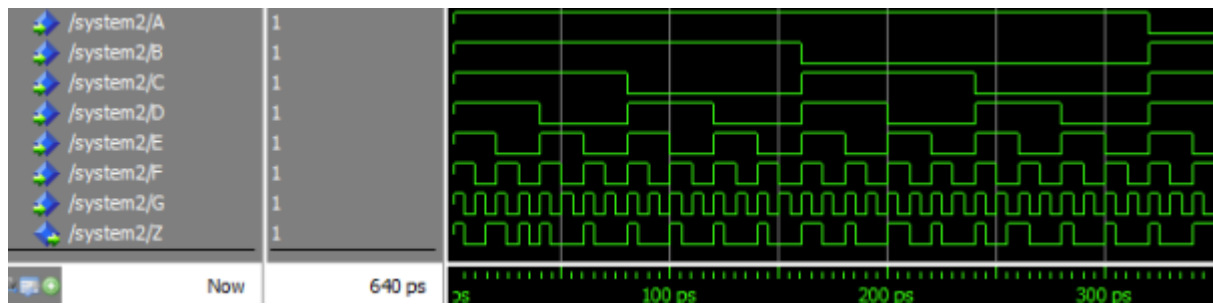


Figura 6: Simulação para A = 1

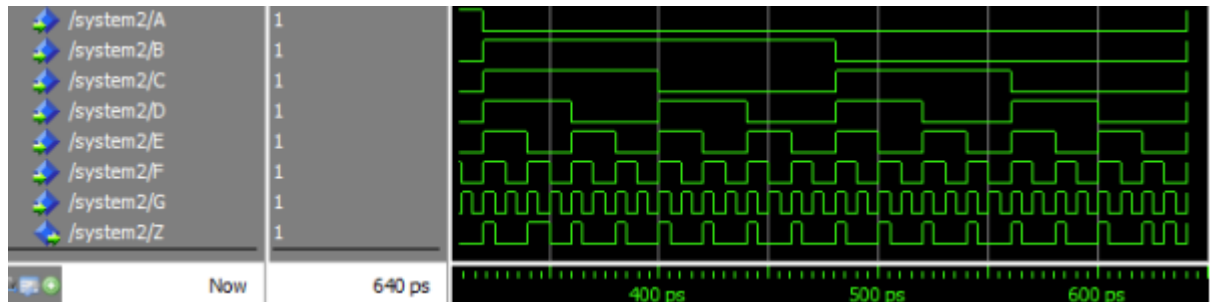


Figura 7: Simulação para A = 0

Portanto ficou igual a tabela verdade apresentada anteriormente na Tabela 3.

CONCLUSÃO

Portanto, neste experimento conseguimos conhecer o método de componentes usados para implementar as funções por meio de circuitos já programados anteriormente com a linguagem VHDL e conseguimos implementar as funções corretamente, conseguindo simulá-las com várias combinações onde todas bateram corretamente.